

JAVA LAB WEEK 6

Experiment Topic

Java Collection Framework – Important Methods of Collection Classes

Required Java Concepts

- Java Collection Framework
 - Generics
 - ArrayList
 - LinkedList
 - Vector
 - Stack
 - HashSet
 - LinkedHashSet
 - TreeSet
 - PriorityQueue
 - ArrayDeque
 - Collection methods such as add(), get(), remove(), size(), push(), pop(), peek(), first(), last(), etc.
-

1. Basic Information

Title: Implementation of Java Collection Framework Classes and Important Methods

2. Objective / Aim

Aim

To write Java programs using different classes of the **Java Collection Framework** and demonstrate their commonly used methods.

Objectives

The programs demonstrate:

- Adding and removing elements from collections.
- Accessing elements using collection methods.

- Working with List, Set, Stack, Queue, and Deque classes.
 - Understanding duplicate handling and ordering in different collections.
 - Using generic collections with Java objects.
-

3. System Requirements

Hardware Requirements

- Computer or Laptop
- Minimum 4 GB RAM
- Keyboard
- Monitor

Software Requirements

- Operating System: Windows / Linux / macOS
 - **JDK 8 or above**
 - Java Compiler: javac
 - IDE: Eclipse / IntelliJ IDEA / NetBeans / VS Code / BlueJ
 - Command Prompt or Terminal
-

4. Algorithm / Logic Description

Program 1: ArrayList, LinkedList and Vector

1. Start the program.
2. Create an ArrayList object.
3. Add elements using add().
4. Access an element using get().
5. Remove an element using remove().
6. Display the number of elements using size().
7. Create a LinkedList object.
8. Add elements using add(), addFirst() and addLast().
9. Display the first element using getFirst().

10. Create a Vector object.
11. Add elements using add().
12. Display its elements and size.
13. Stop the program.

Program 2: Stack, HashSet, LinkedHashSet and TreeSet

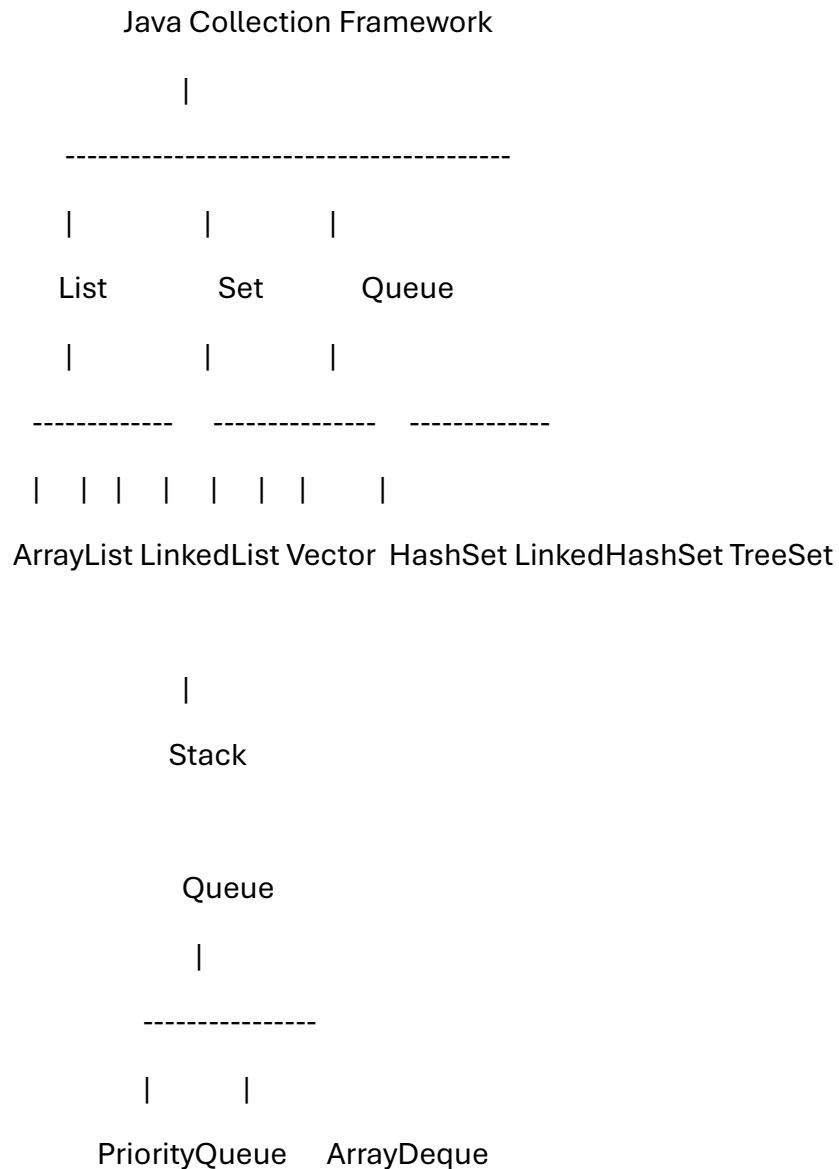
1. Start the program.
2. Create a Stack object.
3. Insert elements using push().
4. Remove the top element using pop().
5. Create a HashSet object.
6. Add elements using add().
7. Add a duplicate element and observe that the duplicate is not stored.
8. Create a LinkedHashSet and add elements.
9. Observe that insertion order is maintained.
10. Create a TreeSet and add elements.
11. Display the set in sorted order.
12. Use first() and last() to obtain the lowest and highest elements.
13. Stop the program.

Program 3: PriorityQueue and ArrayDeque

1. Start the program.
2. Create a PriorityQueue object.
3. Add elements using add().
4. Use peek() to view the head element.
5. Use poll() to remove the head element.
6. Create an ArrayDeque object.
7. Add elements using addFirst() and addLast().
8. Use peekFirst() and peekLast() to view both ends.
9. Display the results.

10. Stop the program.

5. UML Class Diagram



Important Class Features

ArrayList → Resizable array, ordered, duplicates allowed

LinkedList → Doubly linked list, ordered

Vector → Resizable array, synchronized

Stack → LIFO structure

HashSet → No duplicates, unordered

LinkedHashSet → No duplicates, insertion order maintained

TreeSet → No duplicates, sorted order

PriorityQueue → Elements processed according to priority

ArrayDeque → Double-ended queue

6. Source Code

Program 1: ArrayList, LinkedList and Vector

Filename: ListDemo.java

```
import java.util.*;
```

```
class ListDemo {
```

```
    public static void main(String[] args) {
```

```
        ArrayList<String> a = new ArrayList<>();
```

```
        a.add("Java");
```

```
        a.add("C");
```

```
        System.out.println("ArrayList: " + a);
```

```
        System.out.println("Get: " + a.get(0));
```

```
        a.remove("C");
```

```
        System.out.println("Size: " + a.size());
```

```
        LinkedList<String> l = new LinkedList<>();
```

```
        l.add("A");
```

```
        l.addFirst("Start");
```

```
        l.addLast("End");
```

```
        System.out.println("LinkedList: " + l);
```

```
        System.out.println("First: " + l.getFirst());
```

```
        Vector<Integer> v = new Vector<>();
```

```
v.add(10);  
v.add(20);  
System.out.println("Vector: " + v);  
System.out.println("Size: " + v.size());  
}  
}
```

Program 2: Stack, HashSet, LinkedHashSet and TreeSet

Filename: SetDemo.java

```
import java.util.*;
```

```
class SetDemo {
```

```
    public static void main(String[] args) {
```

```
        Stack<Integer> s = new Stack<>();
```

```
        s.push(10);
```

```
        s.push(20);
```

```
        System.out.println("Stack: " + s);
```

```
        System.out.println("Pop: " + s.pop());
```

```
        HashSet<Integer> h = new HashSet<>();
```

```
        h.add(10);
```

```
        h.add(20);
```

```
        h.add(10);
```

```
        System.out.println("HashSet: " + h);
```

```
        LinkedHashSet<String> l = new LinkedHashSet<>();
```

```
        l.add("Java");
```

```
l.add("C");

System.out.println("LinkedHashSet: " + l);


TreeSet<Integer> t = new TreeSet<>();

t.add(30);

t.add(10);

t.add(20);

System.out.println("TreeSet: " + t);

System.out.println("First: " + t.first());

System.out.println("Last: " + t.last());

}

}
```

Program 3: PriorityQueue and ArrayDeque

Filename: QueueDemo.java

```
import java.util.*;

class QueueDemo {

    public static void main(String[] args) {

        PriorityQueue<Integer> p = new PriorityQueue<>();

        p.add(30);

        p.add(10);

        p.add(20);

        System.out.println("PriorityQueue: " + p);

        System.out.println("Peek: " + p.peek());

        System.out.println("Poll: " + p.poll());

    }

}
```

```
    ArrayDeque<Integer> d = new ArrayDeque<>();  
    d.addFirst(10);  
    d.addLast(20);  
    System.out.println("ArrayDeque: " + d);  
    System.out.println("First: " + d.peekFirst());  
    System.out.println("Last: " + d.peekLast());  
}  
}
```

7. Compilation & Execution Commands

Program 1

Java Filename

ListDemo.java

Compilation

javac ListDemo.java

Execution

java ListDemo

Program 2

Java Filename

SetDemo.java

Compilation

javac SetDemo.java

Execution

java SetDemo

Program 3

Java Filename

QueueDemo.java

Compilation

```
javac QueueDemo.java
```

Execution

```
java QueueDemo
```

8. Output Screen

Program 1 Output

ArrayList: [Java, C]

Get: Java

Size: 1

LinkedList: [Start, A, End]

First: Start

Vector: [10, 20]

Size: 2

Explanation

The ArrayList initially contains Java and C. After removing C, its size becomes 1.

The LinkedList contains Start, A, and End, because addFirst() inserts at the beginning and addLast() inserts at the end.

The Vector contains two elements, 10 and 20.

Program 2 Output

Stack: [10, 20]

Pop: 20

HashSet: [20, 10]

LinkedHashSet: [Java, C]

TreeSet: [10, 20, 30]

First: 10

Last: 30

Note

HashSet does **not guarantee insertion order**, so its displayed order may differ on different Java versions or systems. The important point is that the duplicate value 10 is stored only once.

LinkedHashSet maintains insertion order.

TreeSet automatically stores elements in sorted order.

Program 3 Output

PriorityQueue: [10, 30, 20]

Peek: 10

Poll: 10

ArrayDeque: [10, 20]

First: 10

Last: 20

Explanation

The PriorityQueue gives the smallest element as its head, so peek() returns 10 and poll() removes 10.

The ArrayDeque allows insertion and access from both the front and the rear.

9. Result

The Java Collection Framework programs were successfully implemented.

The experiments demonstrated the basic working and important methods of:

ArrayList, LinkedList, Vector, Stack, HashSet, LinkedHashSet, TreeSet, PriorityQueue, and ArrayDeque.

The programs also demonstrated important collection operations such as **adding, accessing, removing, searching, ordering, and retrieving elements.**